

ML course season 2

Кластеризация. Часть II: DBSCAN и плотностной подход

Паша Кашмель

Курс по машинному обучению

4 мая 2026 г.

План лекции

- 1 Повторение
- 2 Плотностная кластеризация
- 3 Параметры `eps` и `min_samples`
- 4 Работа с шумом
- 5 Ограничения DBSCAN и HDBSCAN
- 6 Практика

Что обсудили в прошлый раз

K-Means и его варианты

- Алгоритм: инициализация центроидов → E-шаг → M-шаг
- K-Means++: умная инициализация центроидов
- Mini Batch K-Means: быстро, но шумно
- Bisect K-Means: иерархическое дробление, помогает выбрать K

Метрики качества

- Внутрикластерное расстояние $F_{in} \downarrow$
- Межкластерное расстояние $F_{out} \uparrow$
- Коэффициент силуэта $S(x_i) \in [-1, +1]$

Ключевое ограничение K-Means

Функционал ошибки:

$$L = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \rightarrow \min$$

Штраф за расстояние до центра вынуждает искать **выпуклые, шарообразные** кластеры. Луны, кольца, спирали — недостижимы.

Зачем нужен другой подход?

К-Means плохо работает, когда:

- Кластеры имеют **произвольную форму** (луны, кольца, спирали)
- Данные содержат **выбросы** — шумовые точки
- **Число кластеров** неизвестно заранее
- Кластеры имеют **существенно разную плотность**

Интуиция плотностного подхода

Кластер — это **плотное скопление точек**, окружённое областями с малой плотностью. Если точки «связаны» через плотные области — они в одном кластере.



К-Means не разделит эти луны.
DBSCAN — справится.

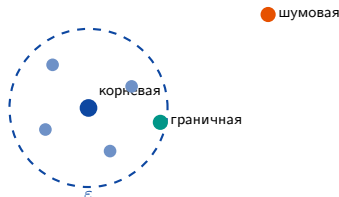
DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Два гиперпараметра:

- ε (eps) — радиус окрестности точки
- `min_samples` — минимум точек в ε -окрестности

Три типа точек:

- **Корневая** (core): в ε -окрестности $\geq \text{min_samples}$ точек
- **Граничная** (border): в ε -окрестности $< \text{min_samples}$, но достижима из корневой
- **Шумовая** (noise): не является ни корневой, ни граничной — метка -1



Шаги алгоритма

- 1 Для каждой **непосещённой** точки найти её ε -окрестность
- 2 Если точек $\geq \text{min_samples}$ — начать **новый кластер**
- 3 **Расширить** кластер: рекурсивно добавить все плотно достижимые точки
- 4 Граничные точки добавить к ближайшей корневой точке
- 5 Оставшиеся точки — **шум** (метка -1)

Ключевое: плотная достижимость

Точка q *плотно достижима* из p , если существует цепочка

$p = p_1, p_2, \dots, p_n = q$, где каждая следующая точка лежит в ε -окрестности предыдущей **корневой** точки.

Именно цепочки через корневые точки позволяют «тянуть» кластер вдоль **произвольных форм** — лун, спиралей, колец.

Влияние параметра ϵ

ϵ слишком мало

- Мало точек попадает в ϵ -окрестность
- Большинство точек становятся **шумом**
- Один реальный кластер дробится на множество мелких

ϵ слишком велико

- Окрестность охватывает почти все точки
- Все кластеры **сливаются** в один
- Шумовые точки поглощаются кластерами

Как выбрать ϵ : k-distance plot

- 1 Для каждой точки вычислить расстояние до k -го ближайшего соседа (где $k = \text{min_samples}$)
- 2 Отсортировать по убыванию и построить график
- 3 Найти «**локоть**» — точку резкого перегиба
- 4 $\epsilon \approx$ значению на перегибе

Идея: в плотном кластере расстояния до k -го соседа малы и примерно равны; в зоне шума они резко растут.

Влияние параметра `min_samples`

Физический смысл: минимальная плотность, при которой точка считается «ядром» кластера.

- **Малые (2–3):** почти все точки корневые, шума мало, но кластеры легко сливаются
- **Большие:** строже требования к плотности, кластеры «чище», больше шума

Практическое правило

Стартовая точка: $\text{min_samples} = 2 \times d$, где d — размерность данных. При зашумлённых данных увеличить.

Взаимосвязь параметров

eps	min_s.	Эффект
мало	мало	много мелких кластеров
мало	много	почти всё — шум
много	мало	один гигантский кластер
много	много	крупные плотные кластеры

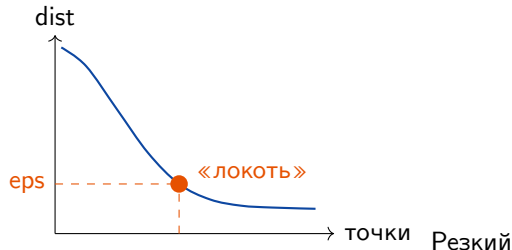
k-Distance Plot на практике

```
from sklearn.neighbors import
↳ NearestNeighbors
import numpy as np, matplotlib.pyplot as plt

k = 5 # обычно = min_samples
nbrs =
↳ NearestNeighbors(n_neighbors=k).fit(X)
distances, _ = nbrs.kneighbors(X)

k_dist = np.sort(distances[:, -1])[:, -1]

plt.plot(k_dist)
plt.xlabel("Точки (отсортированы)")
plt.ylabel(f"Расстояние до {k}-го соседа")
plt.title("k-Distance Plot")
```



перегиб = граница между кластерными точками и шумом

Чувствительность к масштабу признаков

Важно: масштаб имеет значение!

DBSCAN использует евклидово расстояние. Если признаки имеют **разный масштаб**, один признак будет доминировать при вычислении ε -окрестности.

Пример: «возраст» (0–100) и «доход» (0–1 000 000).

Расстояние будет определяться практически только доходом.

Решение: масштабирование

Перед применением DBSCAN **обязательно** нормализовать данные.

```
from sklearn.preprocessing import  
    ↪ StandardScaler  
from sklearn.cluster import DBSCAN  
  
# Масштабирование - обязательный шаг  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)  
  
# Затем DBSCAN  
db = DBSCAN(eps=0.5, min_samples=5)  
labels = db.fit_predict(X_scaled)
```

StandardScaler переводит каждый признак к нулевому среднему и единичной дисперсии: все признаки вносят равный вклад в расстояние.

Метка -1 в DBSCAN означает:

- Точка не принадлежит ни одному кластеру
- В её ϵ -окрестности недостаточно соседей
- Она изолирована от плотных скоплений

Контекст определяет интерпретацию:

- **Очистка данных:** выброс — ошибка измерения
- **Аномалии:** выброс — самый ценный результат
- **Сегментация:** шум можно отнести к ближайшему кластеру

Шум как диагностический сигнал

Если доля шума велика:

- `eps` слишком мало
- `min_samples` слишком велико
- Данные реально содержат аномалии

```
# Анализ шумовых точек
n_noise = (labels == -1).sum()
ratio = n_noise / len(labels)
print(f"Шум: {n_noise} ({ratio:.1%})")

# Координаты выбросов
X_noise = X[labels == -1]
```

Где DBSCAN ломается: разная плотность

Корень проблемы: DBSCAN использует *глобальный* порог плотности ε .

- **Малое ε :** плотный кластер найден хорошо, но разреженный превращается в шум
- **Большое ε :** разреженный кластер найден, но соседние плотные кластеры сливаются



При одном ε : либо разреженный в шум, либо плотные сливаются

Вывод

Для разноплотных данных идеального ε **не существует**. Нужен алгоритм с адаптивным порогом.

HDBSCAN: иерархическое расширение

Ключевая идея

HDBSCAN строит иерархию при всех значениях ϵ и выбирает наиболее *стабильные* кластеры.

Преимущества:

- Кластеры **разной плотности**
- Один параметр: `min_cluster_size`
- Вероятности принадлежности
- Лучше в высокой размерности

Когда выбирать

Разная плотность, высокая d ,
неизвестное K , нужны вероятности.

```
import hdbscan

clusterer = hdbscan.HDBSCAN(
    min_cluster_size=10,
    min_samples=5 # опционально
)
labels = clusterer.fit_predict(X_scaled)

# Вероятности принадлежности [0..1]
probs = clusterer.probabilities_
# Стойкость кластеров
persist = clusterer.cluster_persistence_
```

DBSCAN vs K-Means: итоговое сравнение

Критерий	K-Means	DBSCAN
Форма кластеров	только выпуклые	произвольная
Число кластеров	задаётся вручную (K)	определяется автоматически
Выбросы	нет обработки	метка -1
Вычислительная сложность	$O(N \cdot K \cdot T)$	$O(N \log N)$ с деревом
Разная плотность	плохо	плохо
Детерминизм	нет (зависит от init)	да
Масштабируемость	хорошо	умеренно

Вывод

Нет универсального победителя. Сферические кластеры одинакового размера → K-Means. Произвольные формы с шумом → DBSCAN. Разная плотность → HDBSCAN.

DBSCAN на практике: полный пример

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons
from sklearn.metrics import silhouette_score
import numpy as np

X, _ = make_moons(n_samples=300, noise=0.1)
X_sc = StandardScaler().fit_transform(X)

db = DBSCAN(eps=0.3, min_samples=5)
labels = db.fit_predict(X_sc)

n_cl = len(set(labels)) - (-1 in labels)
n_noise = (labels == -1).sum()
if n_cl > 1:
    mask = labels != -1
    s = silhouette_score(X_sc[mask], labels[mask])
    print(f"Кластеров: {n_cl}, шум: {n_noise}")
    print(f"Силуэт: {s:.3f}")
```

Атрибуты DBSCAN

- labels_ — метки (−1 = шум)
- core_sample_indices_
- components_

Нет predict!

DBSCAN трансдуктивен: не умеет кластеризовать новые точки без переобучения.

Используйте HDBSCAN для онлайн-предсказаний.

Два сценария в ноутбуке

- 1 **2D**: луны и кольца — сравнение DBSCAN с K-Means
- 2 **Wine**: кластеризация в 13D, анализ шума

Что нужно сделать:

- Построить k-distance plot, выбрать ϵ
- Применить DBSCAN и K-Means
- Визуализировать результаты
- Проанализировать шумовые точки
- Сравнить коэффициент силуэта

Вопросы для размышления

- Как меняются результаты при изменении ϵ ?
- Что общего у шумовых точек?
- Корректно ли сравнивать силуэт DBSCAN и K-Means напрямую?

Машинное обучение

Спасибо за внимание!

Вопросы?

Докладчик: Паша Кашмель